# Assets & media



### Asset Bundle
Derlemede pakete eklenir, çalışma zamanında yerelden okunur

### Network Image
Çalışma zamanında HTTP ile indirilir; gecikme ve veri maliyeti taşır

### On-Device Inference
Medya, NPU üzerinde cihaz içinde üretilir; sunucuya istek gitmez

media_source.dart

```dart
enum MediaSource { disk, network, onDevice }

ImageProvider resolve(MediaSource src, String key,
        [Uint8List? bytes]) =>
    switch (src) {
      MediaSource.disk => AssetImage(key),      // bundle
      MediaSource.network => NetworkImage(key), // HTTP
      MediaSource.onDevice => MemoryImage(bytes!), // NPU
    };
```



BENZETME

Bir kargo havalimanını düşünün: Disk, depoda hazır bekleyen konteyner; Ağ, okyanus aşırı yolda olan sevkiyat; NPU ise terminalin içindeki kendi üretim hattı.

# Add assets and images

### pubspec.yaml Konfigürasyonu

Asset'ler assets: listesinde dosya ya da klasör olarak bildirilir

### Asset Bundle Entegrasyonu

Bildirilen her dosya, kodda kullanılmasa bile uygulama boyutuna eklenir

### DevicePixelRatio

AssetImage cihaz DPR'ına en yakın 2.0x / 3.0x varyantını otomatik seçer

pubspec.yaml

```yaml
flutter:
  assets:
    - assets/images/logo.png   # yalnızca ana asset
    - assets/icons/            # klasördeki tüm dosyalar

# Dizin yapısı — varyantlar otomatik paketlenir:
# assets/images/logo.png        1.0x    72 × 72
# assets/images/2.0x/logo.png   2.0x   144 × 144
# assets/images/3.0x/logo.png   3.0x   216 × 216
```

BENZETME

Uçağın yük manifestosuna yazılan her palet kalkışta uçağa biner; içi boş olsa bile yakıt maliyeti ödenir.

# Display images from the internet

**Image.asset**
Asset bundle'dan yerel olarak okunur; ağ erişimi ve gecikme yoktur

**Image.network**
Her istek hücresel radyoyu düşük güç modundan aktif moda geçirir

**Radio Tail Energy**
Transfer bitse de radyo birkaç saniye yüksek güç modunda kalır

network_image.dart

```dart
// Yerel: asset bundle'dan okunur, ağ gecikmesi yok
Image.asset('assets/images/hero.png');

// Ağ: gecikme + radyo aktivasyonu + tail energy
Image.network('https://picsum.photos/250?image=9');

// Image.network yalnızca bellek içi cache kullanır.
// Cache'ten çıkarılan görsel yeniden indirilir;
// hücresel radyo tekrar aktif moda geçer.
```

BENZETME

Havalimanı tek bir uçak için bile tüm pist ışıklarını yakar ve uçak indikten sonra da ışıklar bir süre yanık kalır; fatura, inen yolcu sayısına değil pistin kaç kez açıldığına göre kesilir.

# Fade in images with a placeholder

### FadeInImage
Görsel yüklenene kadar placeholder gösterir, ardından fade geçişi uygular

### Disk Cache
cached_network_image indirilen görseli yerel depolamaya kaydeder

### Offline Fallback
Bağlantı yokken önbellekteki görsel kullanılır; yüklemede Shimmer gösterilir

offline_first.dart

```dart
// docs: placeholder ile fade-in geçişi
FadeInImage.assetNetwork(
  placeholder: 'assets/loading.gif',
  image: 'https://picsum.photos/250?image=9',
);

// Üretim: disk cache + skeleton placeholder
CachedNetworkImage(
  imageUrl: url,
  placeholder: (_, __) => Shimmer.fromColors(
    baseColor: const Color(0xFFE6EEF5),
    highlightColor: Colors.white,
    child: Container(color: Colors.white),
  ),
  errorWidget: (_, __, ___) => const Icon(Icons.cloud_off),
);
```

BENZETME

Radar bağlantısı kesildiğinde hava trafik ekranı kararmaz; uçakların son bilinen konumlarını göstermeye devam eder. Önbellek son bilinen konumdur, Shimmer ise "veri yolda" sinyalidir.

# Resolution-aware image assets



**Decoded Image Size**
RAM'de dosya boyutu değil, decode edilmiş bitmap boyutu yer tutar

**Memory Allocation**
Bitmap = genişlik × yükseklik × 4 bayt (RGBA); 4K görsel ≈ 33 MB

**Out of Memory (OOM)**
Bellek sınırı aşılırsa işletim sistemi uygulama sürecini sonlandırır

oom_math.dart

```dart
// Decode edilmiş görselin RAM maliyeti: piksel × 4 bayt
//   3840 × 2160 × 4 = 33.177.600 bayt ≈ 33 MB

// 120 px avatar, RAM'de 4K bitmap kadar yer tutar:
Image.asset(
  'assets/images/avatar_4k.jpg',
  width: 120,  // render boyutu: 120 px
);             // decode boyutu: 3840 px → 33 MB

// Öneri: asset'i DPR varyantlarıyla hedef boyutta sağla
//   avatar.png 120 px · 2.0x 240 px · 3.0x 360 px
```



BENZETME

Kargo uçağında yükü kutunun boyu değil, kantardaki ağırlığı belirler: 2 MB'lık JPEG, RAM kantarında 33 MB tartar. Sınır aşılırsa pilot kalkışı iptal eder; işletim sistemi ise uygulamayı haber vermeden kapatır.

# Highlight oversized images

**debugInvertOversizedImages**
Gereğinden büyük decode edilen görselleri ters çevirip renklerini tersler

**cacheWidth / cacheHeight**
Görseli ekranda gösterileceği fiziksel piksel boyutunda decode eder

**Memory Footprint**
800 × 450 decode ile 33,2 MB → 1,44 MB (%95,7 azalma)

main.dart

```dart
void main() {
  // yalnızca debug: oversized görselleri işaretler
  debugInvertOversizedImages = true;
  runApp(const App());
}

// 400 mantıksal px × DPR 2.0 = 800 fiziksel px
Image.asset(
  'assets/images/hero_4k.jpg',
  width: 400,
  cacheWidth: 800, // 3840×2160 yerine 800×450
);                 // 33,2 MB → 1,44 MB (%95,7)
```

BENZETME

Konteyner limanında tek bir koli için 40 ayaklık konteyner açılırsa planlama sistemi alarm verir; cacheWidth, yükü gemiye binmeden ihtiyacı kadar kutuya yerleştiren planlayıcıdır.

# Play and pause a video

### video_player Plugin
iOS'ta AVPlayer, Android'de ExoPlayer üzerinden native oynatma yapar

### Hardware Decoding
Video decode işi CPU yerine SoC'deki donanım decoder'ında yürütülür

### Codec Desteği
Donanımın desteklemediği codec yazılım decode'a düşer, CPU yükü artar

video_screen.dart

```dart
_controller = VideoPlayerController.networkUrl(
  Uri.parse(
    'https://flutter.github.io/assets-for-api-docs/'
    'assets/videos/butterfly.mp4',
  ),
);
_initializeVideoPlayerFuture = _controller.initialize();
// Oynat / duraklat
FloatingActionButton(
  onPressed: () => setState(() {
    _controller.value.isPlaying
        ? _controller.pause()
        : _controller.play();
  }),
  child: Icon(_controller.value.isPlaying
      ? Icons.pause : Icons.play_arrow),
);
```

BENZETME

Her aracı durdurup parayı elle sayan gişe memuru CPU'dur; HGS anteni ise ücreti araç geçerken kendi donanımıyla, trafiği durdurmadan keser.

# Create and initialize a VideoPlayerController

### VideoPlayerController
Native player'ı, decoder'ı ve video texture'ını yönetir

### dispose()
Decoder, buffer, texture ve ses oturumu kaynaklarını serbest bırakır

### Resource Leak
dispose edilmeyen controller, sayfa kapansa da kaynakları tutmaya devam eder

video_screen.dart

```dart
@override
void initState() {
  super.initState();
  _controller = VideoPlayerController.networkUrl(
    Uri.parse(videoUrl),
  );
  _initializeVideoPlayerFuture = _controller.initialize();
}

@override
void dispose() {
  _controller.dispose(); // native decoder'ı serbest bırak
  super.dispose();
}
```

BENZETME

Uçak kalktığı halde körük kapıdan çekilmezse o kapı bir sonraki uçağa açılamaz; dispose edilmeyen controller da donanım decoder'ını aynı şekilde işgal eder.

# Build AI-powered apps



**Neural Processing Unit**
ML çıkarımı için optimize edilmiş, watt başına verimli donanım bloğu

**Offline Inference**
Ağ bağlantısı ve sunucu gerektirmez; uçak modunda da çalışır

**Data Privacy**
Girdi verisi ve üretilen medya cihaz dışına çıkmaz

ai_asset_service.dart

```dart
const _npu = MethodChannel('app.media/npu');

Future<Uint8List> generateSticker(String prompt) async {
  final bytes = await _npu.invokeMethod<Uint8List>(
    'generate',
    {'prompt': prompt, 'size': 512},
  );
  return bytes!; // ağ bağlantısı gerektirmez
}

// Slayt 1'deki üçüncü kaynak: on-device üretilen medya
final png = await generateSticker('suluboya dağ manzarası');
Image.memory(png, gaplessPlayback: true);
```



BENZETME

Uzaktaki santralden enerji beklemek yerine kendi enerjisini üreten mikro şebeke gibi: Şehir şebekesi çökse de (uçak modu) hastanenin ışıkları yanmaya devam eder.

# Write platform-specific code

### Native API Erişimi
Core ML ve Neural Engine API'leri Swift tarafından çağrılır

### MethodChannel
Dart'tan platform koduna method çağrısı ve parametre iletir, sonucu döndürür

### Asenkron Mesajlaşma
Mesajlar asenkron iletilir; çağrı sürerken UI thread bloklanmaz

AppDelegate.swift

```swift
let npu = FlutterMethodChannel(name: "app.media/npu",
    binaryMessenger: controller.binaryMessenger)
npu.setMethodCallHandler { call, result in
  guard call.method == "generate" else {
    return result(FlutterMethodNotImplemented) }
  let config = MLModelConfiguration()
  config.computeUnits = .cpuAndNeuralEngine  // ANE
  Task { @MainActor in
    do {
      let png = try await StickerModel.generate(
          call.arguments, config: config)
      result(FlutterStandardTypedData(bytes: png))
    } catch { result(FlutterError(code: "NPU_FAIL",
                  message: "\(error)", details: nil)) }
  }
}
```

BENZETME

Fly-by-wire uçakta pilotun kolu kanatçığa mekanik olarak bağlı değildir; komut elektrik sinyaline dönüşür ve uçuş bilgisayarı onu eyleyicilere iletir. Dart pilot, MethodChannel kablo, Swift ve Kotlin eyleyicidir.

# Rendering performance

### Frame Budget
60 FPS'te kare başına 16,7 ms, 120 Hz ekranda 8,3 ms

### Thermal Throttling
SoC ısındığında işletim sistemi CPU/GPU saat frekansını düşürür

### Thermal State API
thermalState kritik seviyedeyken inference ertelenir, UI önceliklenir

thermal_guard.dart

```dart
const _thermal = MethodChannel('app.device/thermal');

// iOS:     ProcessInfo.thermalState
//          nominal · fair · serious · critical
// Android: PowerManager.currentThermalStatus
//          NONE … SHUTDOWN
Future<void> thermalAwareGenerate(String prompt) async {
  final state = await _thermal.invokeMethod<String>('get');
  if (state == 'serious' || state == 'critical') {
    _queue.add(prompt); // inference'ı ertele, UI'ı koru
    return;
  }
  await generateSticker(prompt);
}
```

BENZETME

Şehir şebekesi aşırı yüklenince dağıtım şirketi hastaneyi değil, planlı olarak sıradaki mahalleyi keser; iyi tasarlanmış uygulama da ısınınca arayüzü değil, arka plandaki üretimi keser.

# Debugging tools



**Problem**
Görsel listesi kaydırıldıkça cihaz ısınıyor ve uygulama kapanıyor

**Veri Seti**
300 görsel, her biri 4000 × 3000 px; ekranda yalnızca 200 px yüksekliğinde

**İnceleme Noktası**
Image.network decode boyutunu ve disk cache'i nasıl yönetiyor?

gallery_bad.dart

```dart
// 300 adet 12 MP fotoğraf,
// ekranda her biri 200 px yükseklikte
ListView(
  children: [
    for (final url in photoUrls)
      Image.network(
        url,
        height: 200,
        fit: BoxFit.cover,
      ),
  ],
);

// VAKA: Cihaz neden ısınıyor, uygulama neden sonlanıyor?
```



BENZETME

Bir şehrin tüm trafik kameralarını 4K yayına ayarlayıp görüntüyü hiç kaydetmeden her bakışta merkezden yeniden çekmek gibi: hat tıkanır, sunucu odası ısınır, sonunda şalter atar.

# Work with long lists

### ListView.builder
Yalnızca görünür alandaki öğeler build edilir, gerisi lazy yüklenir

### memCacheWidth
Görsel ekran genişliğinde decode edilir: 48 MB → ~4 MB (%91 azalma)

### Disk Cache
İndirilen görsel diskte saklanır; tekrar gösterimde ağ isteği yapılmaz

gallery_fixed.dart

```dart
// ÇÖZÜM
ListView.builder(
  itemCount: photoUrls.length,
  itemBuilder: (context, i) {
    final px = MediaQuery.sizeOf(context).width *
        MediaQuery.devicePixelRatioOf(context); // 1170
    return CachedNetworkImage(
      imageUrl: photoUrls[i],
      height: 200,
      fit: BoxFit.cover,
      memCacheWidth: px.round(), // 4000 → 1170 px
      maxWidthDiskCache: 1200,   // disk cache boyut sınırı
      placeholder: (_, __) => const ShimmerTile(),
    );
  },
);
```

BENZETME

Modern bir dağıtım merkezi tüm stoğu rampaya yığmaz: yalnızca sıradaki kamyonun yükünü kasasına göre paketler, stoğu yerel depoda tutar ve aynı ürün için fabrikaya ikinci kez sipariş geçmez.

# Memory view

### Problem
Video sayfası her açılıp kapandığında RAM ve batarya tüketimi artıyor

### Root Cause
dispose() çağrılmadığı için her ziyarette bir native player açık kalıyor

### Memory view
İki heap snapshot karşılaştırılarak sızan controller örnekleri bulunur

video_page.dart

```dart
// VAKA: her ziyarette yeni player oluşur, hiçbiri kapanmaz
@override
void initState() {
  super.initState();
  _controller = VideoPlayerController.networkUrl(
    Uri.parse(url),
  )..initialize().then((_) => _controller.play());
}
// dispose() yok → ses, decoder ve buffer açık kalır

// ÇÖZÜM: üç satır
@override
void dispose() {
  _controller.dispose();
  super.dispose();
}
```

BENZETME

Şehir su şebekesinde kapatılmayan tek bir vana küçük bir sızıntıdır; binlercesi birikince basınç düşer ve sayaç durmadan döner. Memory view, o vanaları gösteren kaçak tespit haritasıdır.

Dinlediğiniz için teşekkür ederim